作品名　PINKSTAR　(ピンクスター)

ジャンル　2Dスクロールアクション

開発環境　C++,DxLib

対応機種　windows

制作期間　3か月

制作時期　2025年11月～

制作人数　1人



ゲーム概要

吸い込み＆はき出しの
アクションで進んでいく



ボスを倒したらクリア



技術紹介①
ステートパターン

プレイヤーの状態遷移が複雑に
なってしまう

⇩

状態ごとに処理を分離できるように
基底クラスを作り、それを継承して
各状態クラスを作る

⇩

状態管理がわかりやすくなり、
状態の追加もスムーズに行えた

```cpp
class StateBase
{
public:
    virtual ~StateBase() = default;
    /// <summary>
    /// その状態になった時、１回だけ呼ばれる関数
    /// </summary>
    /// <param name="player">プレイヤーの参照</param>
    virtual void Enter(Player& player) = 0;
    /// <summary>
    /// その状態のとき毎フレーム呼ばれる関数
    /// </summary>
    /// <param name="player">プレイヤーの参照</param>
    /// <param name="input">入力を受け取るためのInputの参照</param>
    virtual void Update(Player& player, Input& input) = 0;
    /// <summary>
    /// その状態が終わる時、１回だけ呼ばれる関数
    /// </summary>
    /// <param name="player">プレイヤーの参照</param>
    virtual void Exit(Player& player) = 0;
    /// <summary>
    /// 現在のステートを取得する関数
    /// </summary>
    /// <returns>現在のステート</returns>
    virtual PlayerStateType GetState() const = 0;
};
```

```cpp
    //現在の状態のUpdateを呼び出す
    state_->Update(*this, input);
```

技術紹介②
入力の抽象化

キーボードとPAD両方の入力処理を記述するの面倒

⇩

「左」「右」などのイベント名で入力を取得するクラスを作成

⇩

デバイスごとの処理を記述する必要がなくなった

```cpp
class Input
{
private:
    //イベント名と実際の入力の対応表
    std::map<std::string, std::vector<InputState>>inputTable_;
    //実際に入力されたかどうかのデータ
    std::map<std::string, bool>inputData_;
    //前のフレームに押されたかどうか
    std::map<std::string, bool>lastInputData_;
public:
    Input();
    /// <summary>
    /// 入力情報の更新
    /// </summary>
    void Update();
    /// <summary>
    /// 特定のボタンが押されているか
    /// </summary>
    /// <param name="name">イベント名</param>
    /// <returns>押されている:true / 押されていない:false</returns>
    bool IsPressed(const char* name)const;
    /// <summary>
    /// 特定のボタンが今押されたか(押された瞬間)
    /// </summary>
    /// <param name="name">イベント名</param>
    /// <returns>今押された:true / 押されていないor押しっぱなし:false</returns>
    bool IsTriggered(const char* name)const;
};
```

```cpp
inputTable_["left"] = { {PeripheralType::keyboard,KEY_INPUT_LEFT},
                        {PeripheralType::pad1,PAD_INPUT_LEFT} };
inputTable_["up"] = { {PeripheralType::keyboard,KEY_INPUT_UP},
                      {PeripheralType::pad1,PAD_INPUT_UP} };
inputTable_["right"] = { {PeripheralType::keyboard,KEY_INPUT_RIGHT},
                         {PeripheralType::pad1,PAD_INPUT_RIGHT} };
inputTable_["down"] = { {PeripheralType::keyboard,KEY_INPUT_DOWN},
                        {PeripheralType::pad1,PAD_INPUT_DOWN} };

inputTable_["jump"] = { {PeripheralType::keyboard,KEY_INPUT_SPACE},
                        {PeripheralType::pad1,PAD_INPUT_B} };

inputTable_["attack"] = { {PeripheralType::keyboard,KEY_INPUT_Z},
                          {PeripheralType::pad1,PAD_INPUT_A} };
```